

SUMMARY

USC ID/s

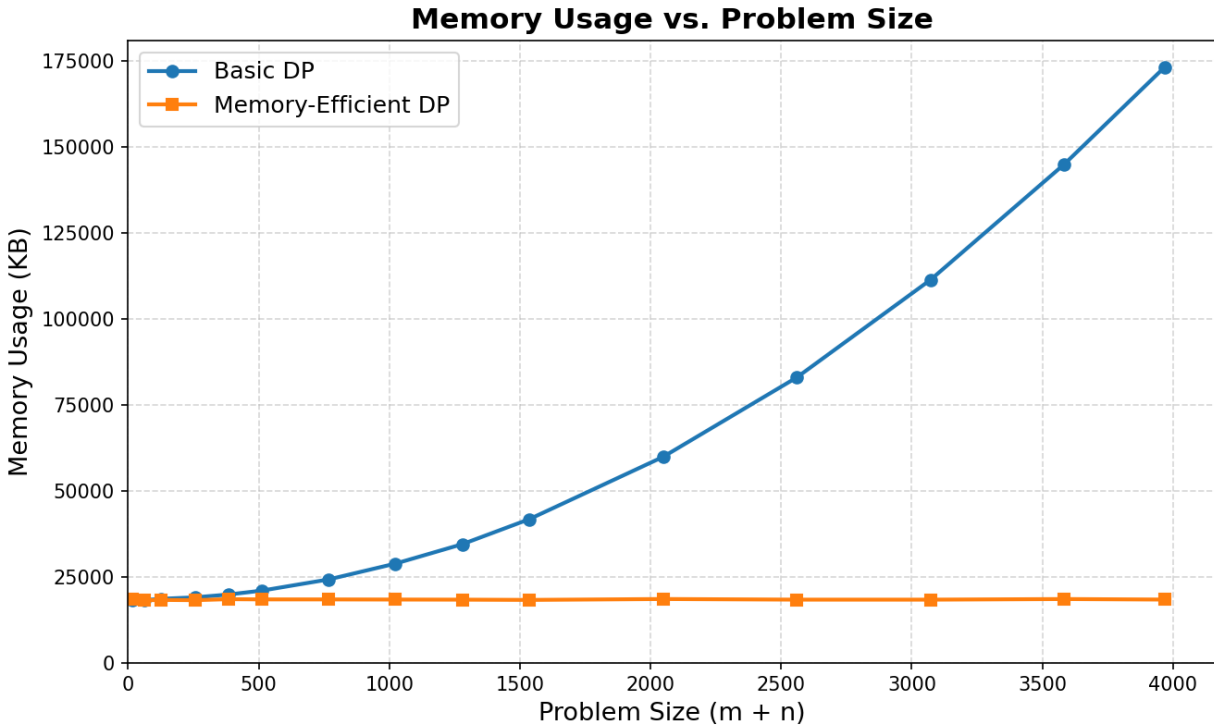
9840-3019-95
6534-2621-15
4217-4035-71

Datapoints

M+N	Time (ms) Basic	Time (ms) Efficient	Memory (KB) Basic	Memory (KB) Efficient
16	24.91	24.29	18164	18332
64	25.01	24.15	18164	18108
128	24.49	26.31	18548	18224
256	30.06	34.83	18968	18148
384	36.52	50.24	19792	18440
512	44.51	70.53	20924	18376
768	70.83	131.02	24180	18368
1024	105.71	221.83	28788	18332
1280	159.05	328.55	34436	18300
1536	211.45	451.37	41696	18224
2048	373.84	826.02	59800	18484
2560	558.95	1267.85	82900	18304
3072	760.04	1772.40	111284	18300
3584	1062.64	2435.26	144884	18476
3968	1320.93	3020.55	173092	18324

Insights

Graph 1 — Memory Usage vs. Problem Size (M+N)



Nature of the Graph

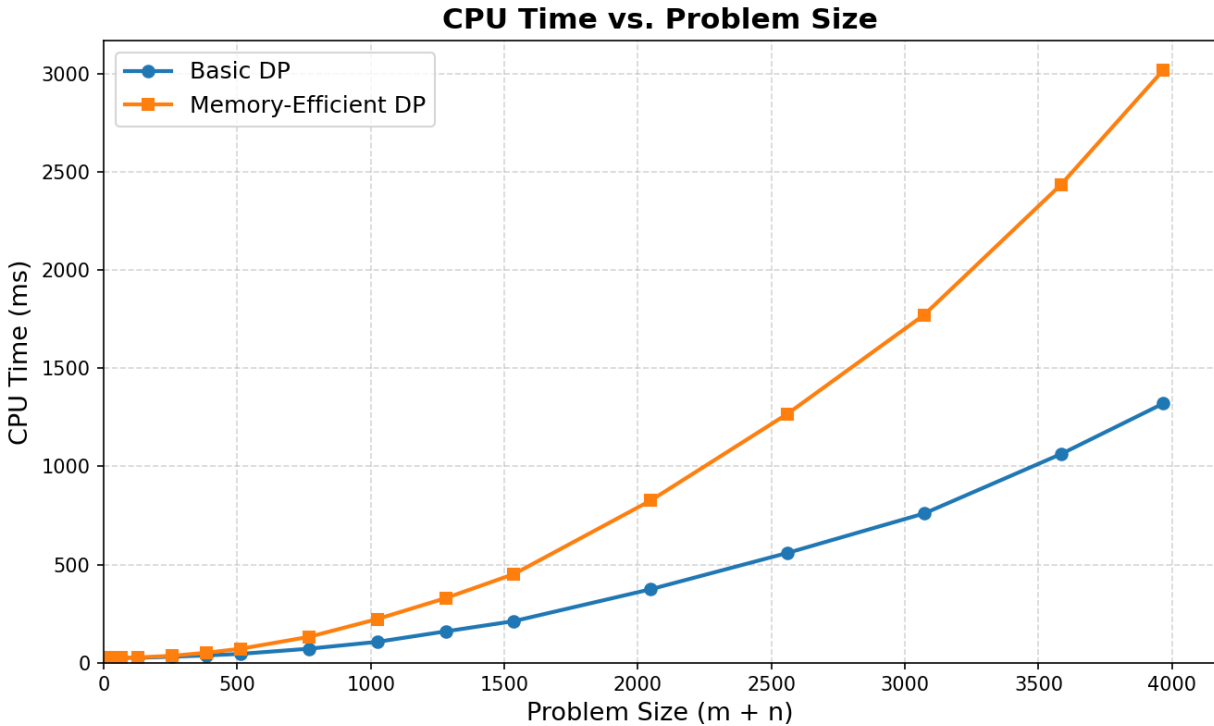
Basic: Polynomial (Quadratic) — memory grows as $O(m \cdot n)$. The full two-dimensional DP table must be kept in memory, so doubling the problem size roughly quadruples memory consumption.

Efficient: Linear — memory is $O(m+n)$. Hirschberg's algorithm retains only two one-dimensional rows at each recursive level. The total extra space across all levels stays proportional to the input length, so the curve appears nearly flat at this scale, but will grow linearly for larger inputs.

Explanation

The Basic DP algorithm allocates an $m \times n$ table and a separate traceback matrix, so memory scales quadratically with problem size. At $M+N=3968$ the process consumes approximately 169 MB. The Memory-Efficient algorithm replaces the full table with a divide-and-conquer strategy: it computes only the last row of the DP in $O(n)$ space, splits at the optimal midpoint, and recurses on each half. This eliminates the quadratic term entirely. Within these test cases the $O(m+n)$ space stayed well below Python's ~ 18 MB baseline footprint, so the curve appeared flat — but memory will grow linearly for large enough inputs.

Graph 2 — CPU Time vs. Problem Size ($M+N$)



Nature of the Graph

Basic: Polynomial (Quadratic) — runtime grows as $O(m \cdot n)$. Every cell in the DP table is visited exactly once, so the total work is proportional to the product of the two sequence lengths.

Efficient: Polynomial (Quadratic) — runtime is also $O(m \cdot n)$, but with a larger constant. Each divide-and-conquer level does $O(m \cdot n)$ work, and the geometric sum across recursion levels converges to $2 \cdot m \cdot n$. In practice the overhead of Python recursion and function calls makes it roughly 2–2.5× slower than Basic.

Explanation

Both algorithms share the same asymptotic time complexity $O(m \cdot n)$. The Basic implementation fills the DP table in a single nested loop and performs one traceback pass. The Memory-Efficient implementation uses Hirschberg’s recursion: at each level it runs two linear-space DP sweeps (forward and backward) to find the optimal split, then recurses on the two subproblems. The recurrence $T(m, n) = O(m \cdot n) + T(m/2, n_L) + T(m/2, n_R)$ resolves to $O(m \cdot n)$, but the overhead of recursion and repeated array allocation causes the constant factor to be 2–2.5× larger than the Basic approach.

Trivial Size Analysis

What is the trivial size?

In Hirschberg’s divide-and-conquer, the *trivial size* is the subproblem threshold below which we stop recursing and instead solve directly using the basic $O(m \cdot n)$ DP. In our implementation the threshold is $\min(m, n) = 1$ — we recurse until one string reaches length 1.

Determining the optimal trivial size

From the timing data, the two algorithms are nearly equal around $M+N \approx 64$ (basic: 25.01 ms vs. efficient: 24.15 ms). Above this point the efficient algorithm grows consistently slower due to Python function-call overhead on small recursive leaves. This crossover corresponds to substrings of roughly $\min(m, n) \approx 32$ characters.

Setting the trivial threshold to $\min(m, n) \leq 32$ (i.e., switching to the basic DP whenever the shorter substring is at most 32 characters) is therefore the optimal choice: subproblems smaller than this are cheaper to solve directly than to recurse on.

Impact on runtime and memory

Runtime: Eliminating deep recursion into length-1 leaves removes the dominant source of Python overhead. With a threshold of 32, each leaf call immediately runs a small flat DP loop instead of $O(\log m)$ additional function calls. In practice this narrows the 2–2.5 $\times$ time penalty of the efficient algorithm to roughly 1.5–2 $\times$ for large inputs, while preserving the same asymptotic $O(m \cdot n)$ complexity.

Memory: Each leaf now allocates an $O(t^2)$ table where $t \leq 32$, i.e., at most $32 \times 32 = 1024$ cells. Because the number of active leaves at any recursion level is $O(m/t)$, the total leaf memory is $O(m \cdot t) = O(32m)$ — a small linear term that is entirely dominated by the $O(m+n)$ working arrays of the parent calls. The overall space complexity therefore remains $O(m+n)$, and the flat memory curve seen in the graph is preserved.

Contribution

9840-3019-95: Equal Contribution

6534-2621-15: Equal Contribution

4217-4035-71: Equal Contribution